

SOB SOLUTIONS · BRAND-ALIGNED REPORT

Influência de padrões numéricos (int8 vs float64) em rede neural MLP embarcada de pouso aeronáutico

Disciplina de Sistemas Operacionais · FIAP
Prof. Dr. José Gomes Salim-Neto · Semestre 1 / 2026

Tá sob controle.

EQUIPE

Matheus Farias de Lima — RM554254
Miguel Mauricio Parrado Patarroyo — RM554007
Vitor Pinheiro Nascimento — RM553693
Gabriel Leão — RM552642
Pedro Henrique Nardaci Chaves — RM553988

Questão de pesquisa, objetivo geral e objetivos específicos

Questão de pesquisa. Trocar float64 por int8 num sistema embarcado de pouso faz a rede neural errar mais? E quanto isso custa em tempo de processamento? Essa é a pergunta que queremos responder. O cenário é um instrumento de bordo que precisa estimar, durante o pouso, quanto de pista ainda resta até a aeronave parar.

Objetivo geral. Medir o quanto o padrão numérico usado nos cálculos da rede muda a qualidade da previsão e o tempo de treinamento. Para isso treinamos a mesma MLP duas vezes, uma em float64 e outra em int8, e comparamos.

Como chegamos lá

Primeiro modelamos a física do pouso e usamos essas equações para gerar o dataset, já que aceleração de pouso não é constante e isso importa para a qualidade dos dados. Em seguida exploramos e preparamos esses dados, implementamos a MLP do zero para ter controle total sobre a precisão numérica, treinamos nos dois padrões medindo épocas, erro e tempo, e por fim discutimos o papel da pré-empção do sistema operacional sobre o que observamos.

02 — CINEMÁTICA DO POUSO

Equações cinemáticas do processo de pouso

Quando a aeronave toca a pista, três forças atuam contra o movimento: o arrasto do ar, o empuxo reverso das turbinas e a frenagem das rodas. O detalhe importante é que o arrasto cresce com o quadrado da velocidade e o reverso cresce com a velocidade, então a desaceleração muda o tempo todo durante o pouso. Ou seja, não é aceleração constante e as fórmulas de M.U.V. não servem aqui. Pela segunda lei de Newton:

$$m \cdot (dv/dt) = - F_{\text{frenagem}} - F_{\text{arrasto}}(v) - F_{\text{reverso}}(v)$$

Cada força é:

$$F_{\text{arrasto}}(v) = \frac{1}{2} \cdot C_d \cdot A \cdot v^2$$

$$F_{\text{reverso}}(v) = k_{\text{rev}} \cdot v$$

$$F_{\text{frenagem}} = \mu \cdot m \cdot g$$

Juntando tudo, a velocidade obedece a uma equação diferencial não linear:

$$dv/dt = - [\frac{1}{2} C_d A \cdot v^2 + k_{\text{rev}} \cdot v + \mu m g] / m$$

A posição é a integral da velocidade. Como a aceleração depende da própria velocidade, não dá pra resolver com uma conta direta, então integramos numericamente por Runge-Kutta de 4ª ordem com passo de 0,05 s. O gráfico abaixo mostra a aceleração variando ao longo do pouso, confirmando que o movimento é mesmo variado de forma não uniforme.

Perfil físico de um pouso (movimento variado não uniformemente)

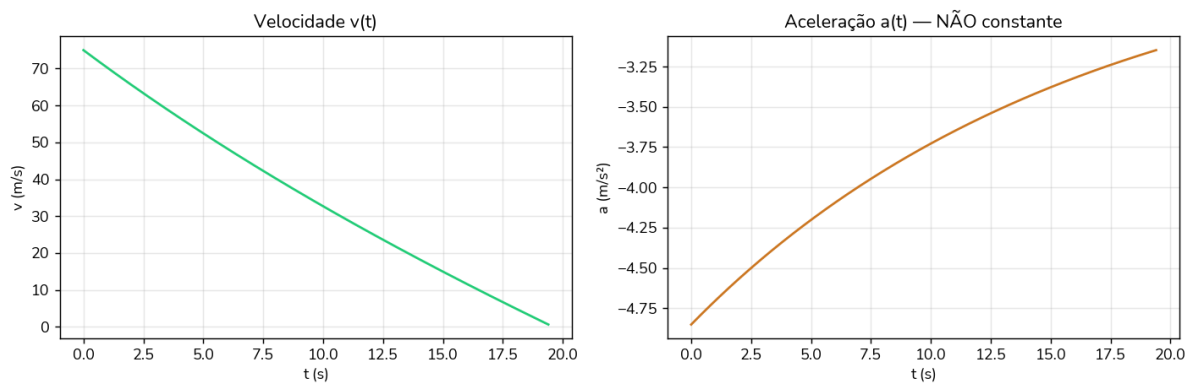


Figura 1 — Velocidade e aceleração de um pouso simulado. A aceleração vai de cerca de $-4,9$ a $-3,1$ m/s^2 .

03 — DADOS

Critérios de arrumação dos dados de pouso

Os dados são sintéticos, gerados a partir do modelo físico da seção anterior. Simulamos milhares de pousos variando massa da aeronave, coeficiente de arrasto, área de referência, atrito da pista e força do reverso, o que dá 6 milhões de linhas com 5 atributos cada.

Atributo	Descrição	Unidade
t	tempo desde o toque	s
v	velocidade instantânea	m/s
x	distância percorrida na pista	m
a	aceleração instantânea	m/s ²
massa	massa da aeronave no pouso	kg
dist_restante (alvo)	distância de pista até parar	m

Na exploração olhamos média, desvio e extremos de cada coluna para conferir se fazia sentido fisicamente: velocidade caindo de uns 75 m/s até parar, distância restante diminuindo até zero e aceleração sempre negativa. No tratamento tiramos valores infinitos ou ausentes e descartamos simulações curtas demais para serem úteis.

Uma decisão importante na preparação: treinamos a rede numa amostra aleatória de 40 mil linhas, e não nas 6 milhões. Rodar a MLP que escrevemos em NumPy sobre o dataset inteiro a cada época seria pesado demais para o tempo que tínhamos, e 40 mil linhas já cobrem bem a variedade de pousos. Por fim normalizamos as features para o intervalo de 0 a 1, o que a sigmoide pede e ainda deixa a faixa controlada para a quantização int8 funcionar. Separamos 80% para treino e 20% para teste.

04 — RESULTADOS

Resultados do treinamento: int8 vs float64

A rede tem uma camada oculta de 16 neurônios e sigmoide nas duas camadas. Treinamos com gradiente descendente e parada antecipada, que interrompe o treino quando o erro para de melhorar. A tabela traz os números dos dois padrões.

Padrão	Épocas	MSE	MAE	R ²	Tempo (min)
float64	1500	0.00255	0.0390	0.9397	0.2475
int8	1407	0.00291	0.0397	0.9312	0.2811

Tabela 2 — Épocas, erro e tempo de processamento de cada padrão. Métricas sobre o alvo normalizado.

Vale ser honesto sobre o tamanho da diferença: o float64 ficou melhor, mas por pouco, com R² de 0,94 contra 0,93 do int8. A relação entre os dados e a resposta é simples o bastante para que a perda de precisão do int8 não derrube muito o resultado. O efeito aparece mais na forma da curva de treino do que no número final.

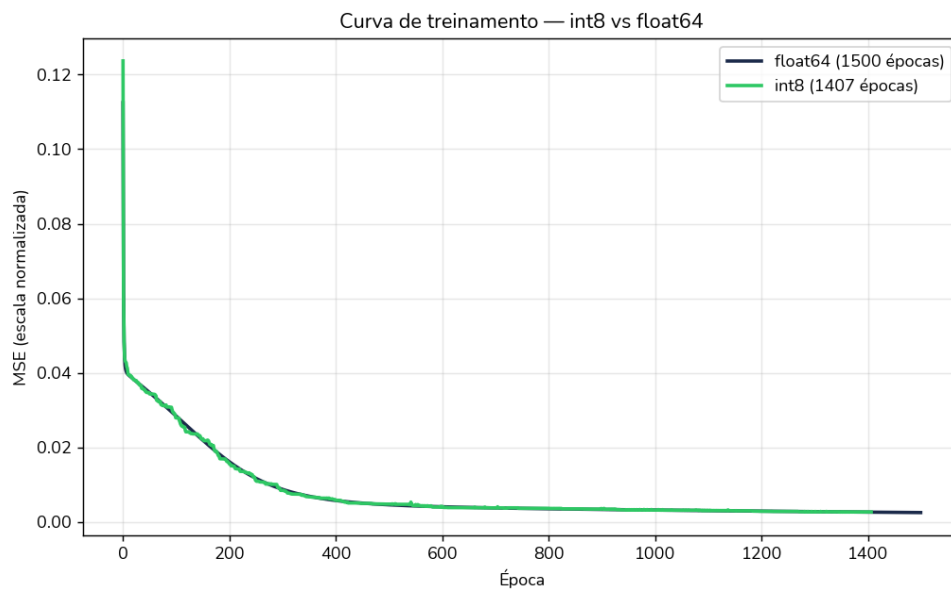


Figura 2 — Erro ao longo das épocas. A curva do int8 treme mais, por causa do ruído de quantização, e para antes pela parada antecipada.

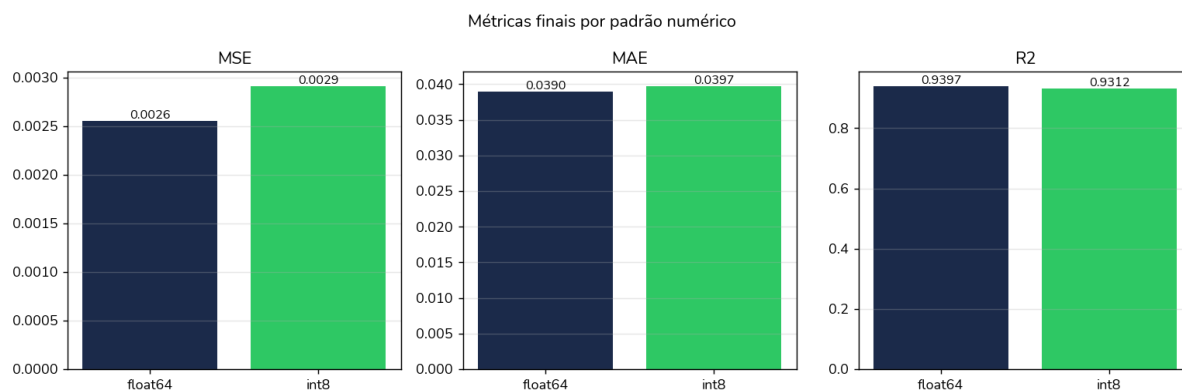


Figura 3 — Erro e R^2 finais. O float64 leva uma pequena vantagem nas três métricas.

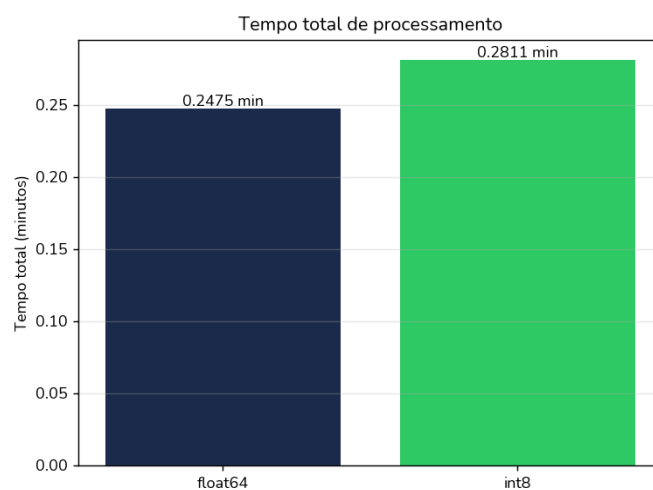


Figura 4 — Tempo total. Em software o int8 saiu mais lento, porque cada passo ainda paga as contas de quantização.

Influência da pré-empção do sistema operacional

Um sistema operacional pré-emptivo pode parar uma tarefa no meio para dar a CPU a outra mais urgente. Num instrumento de avião, que é um ambiente de tempo real, isso pesa de dois jeitos sobre o que medimos aqui.

Pelo lado que atrapalha: como o sistema fica intercalando tarefas, o tempo que medimos não depende só da conta que a rede faz, mas também das trocas de contexto e da espera pela CPU. As contas em int8 deveriam ser mais leves no hardware, mas na nossa implementação elas ainda carregam os passos de quantização, e sob pré-empção esses passos podem ser picados em mais pedaços. Isso ajuda a explicar por que o int8 não saiu mais rápido como se esperaria.

Pelo lado que ajuda: é justamente a pré-empção que garante que a estimativa de pista, que é a tarefa crítica, receba a CPU na hora certa, passando na frente de rotinas menos importantes. Como o int8 ocupa menos memória e cada operação é mais barata, essa tarefa tem mais chance de caber numa única fatia de tempo do escalonador, o que reduz o risco de estourar um prazo. O preço disso é a precisão um pouco menor que vimos nos resultados.

No fim, é uma escolha entre duas coisas: o float64 prevê melhor, o int8 é mais enxuto e se encaixa melhor num escalonador de tempo real. Num sistema embarcado de verdade, decidir entre os dois é equilibrar o quanto de exatidão você precisa contra a garantia de responder dentro do prazo.

Missão cumprida — tá sob controle.